DSA Assignment

UNIT 1:

1. A company stores employee IDs in ascending order. Write a C program that accepts n employee IDs, searches for a required ID using Binary Search, displays its position when found, reports when it is absent, and counts the number of comparisons. Test the program for both successful and unsuccessful searches.

Source code :-

```
#include <stdio.h>

int main(){
   int a[100], n, key;
   int low, high, mid, comparisons = 0;
   int found = 0;
   printf("Enter number of employees: ");
   scanf("%d", &n);
   printf("Enter employee IDs in ascending order:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &a[i]);
   printf("Enter employee ID to search: ");
   scanf("%d", &key);
   low = 0;
   high = n - 1;
   while (low <= high)
   {
      mid = (low + high) / 2;
      comparisons++;
      if (a[mid] == key)
      {
         printf("Employee ID found at position %d\n", mid + 1);
         found=1;
  break;
      }
```

```
        else if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    if (found == 0)
        printf("Employee ID not found\n");
    printf("Number of comparisons: %d\n", comparisons);
    return 0;
}
```

Output :-

```
Enter number of employees: 5
Enter employee IDs in ascending order:
1 2 3 4 5
Enter employee ID to search: 3
Employee ID found at position 3
Number of comparisons: 1
```

```
Enter number of employees: 4
Enter employee IDs in ascending order:
101 103 105 107
Enter employee ID to search: 9
Employee ID not found
Number of comparisons: 2
```

2. A teacher wants to arrange student marks in ascending order and also measure how much

rearrangement is necessary. Write a C program using Insertion Sort that accepts n marks, displays the array after every pass, counts the total number of element shifts, and displays the final sorted list and shift count.

Sourcecode :-

```
#include <stdio.h>
int main()
{
    int a[100], n, i, j, key;
    int shifts = 0;
    printf("Enter number of students: ");
    scanf("%d", &n);
    printf("Enter student marks:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 1; i < n; i++)
```

```
{
        key = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > key)
        {
            a[j + 1] = a[j];
            j--;
            shifts++;
        }
        a[j + 1] = key;
        printf("Pass %d: ", i);
        for (int k = 0; k < n; k++)
        {
            printf("%d ", a[k]);
        }
        printf("\n");
    }
    printf("\nFinal sorted list: ");
    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\nTotal number of shifts: %d\n", shifts);
    return 0;
}
```

Output :-

```
Enter number of students: 5
Enter student marks:
100 99 97 98 95
Pass 1: 99 100 97 98 95
Pass 2: 97 99 100 98 95
Pass 3: 97 98 99 100 95
Pass 4: 95 97 98 99 100

Final sorted list: 95 97 98 99 100
Total number of shifts: 9
```

UNIT 2:

3. An expression-processing application receives an arithmetic expression in infix form. Write a C program using a stack to convert it to postfix form while correctly handling parentheses and operator precedence for +, -, *, / and ^. Test the program using an expression containing multiple operators and parentheses.

Source code :-

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>

char stack[100];

int top = -1;

void push(char c){

   stack[++top] = c;

}

char pop(){

   return stack[top--];

}

int precedence(char c)

{

   if (c == '^')

      return 3;

   if (c == '*' || c == '/')

      return 2;

   if (c == '+' || c == '-')

      return 1;

   return 0;

}

int main()

{

   char infix[100], postfix[100];

   int i, j = 0;

   char c;
```

```
printf("Enter infix expression: ");
scanf("%s", infix);
for (i = 0; i < strlen(infix); i++)
{
   c = infix[i];
   if (isalnum(c))
   {
      postfix[j++] = c;
   }
   else if (c == '(')
   {
      push(c);
   }
   else if (c == ')')
   {
      while (top != -1 && stack[top] != '(')
      {
         postfix[j++] = pop();
      }
      if (top != -1)
         pop();
   }
   else
   {
      while (top != -1 &&
           stack[top] != '(' &&
           (precedence(stack[top]) > precedence(c) ||
           (precedence(stack[top]) == precedence(c) && c != '^')))
      {
         postfix[j++] = pop;
```

```
}
  push(c);
      }
   }
   while (top != -1)
   {
      postfix[j++] = pop();
   }
   postfix[j] = '\0';
   printf("Postfix expression: %s\n", postfix);
   return 0;
}
```

Output :-

```
Enter infix expression: a+b*(c-d)^e
Postfix expression: abcd-e^*+
```

4. A service centre uses a fixed-size request buffer in which released positions must be reused. Write a C program to implement a Circular Queue using an array with insertion, deletion, display, overflow and underflow operations. Demonstrate that positions freed after deletion can be reused for new requests.

Source code :-

```
#include <stdio.h>

#define SIZE 5

int queue[SIZE];

int front = -1, rear = -1;

void enqueue(){
   int value;
   if ((rear + 1) % SIZE == front) {
      printf("Queue Overflow\n");
      return;
   }
```

```
    printf("Enter request: ");
    scanf("%d", &value);
    if (front == -1)
        front = rear = 0;
    else
        rear = (rear + 1) % SIZE;
    queue[rear] = value;
    printf("Request inserted\n");
}

void dequeue()
{
    if (front == -1)
    {
        printf("Queue Underflow\n");
        return;
    }
    printf("Deleted request: %d\n", queue[front]);
    if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % SIZE;
}
void display()
{
    int i;
    if (front == -1)
    {
        printf("Queue is empty\n");
        return;
    }
```

```
    printf("Queue: ");
    i = front;
    while (1)
    {
        printf("%d ", queue[i]);
        if (i == rear)
            break;
        i = (i + 1) % SIZE;
    }
    printf("\n");
}
int main()
{
    int choice;
    while (1)
    {
        printf("\n--- Circular Queue ---\n");
        printf("1. Insert\n");
        printf("2. Delete\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                enqueue();
                break;
            case 2:
                dequeue();
```

```
            break;
        case 3:
            display();
            break;
        case 4:
            return 0;
        default:
            printf("Invalid choice\n");
      }
   }
   return 0;
}
```

Output :-

```
--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter request: 5
Request inserted

--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter request: 0
Request inserted

--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 1
Enter request: 5
Request inserted
```

```

--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
Queue: 5 0 5

--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 2
Deleted request: 5

--- Circular Queue ---
1. Insert
2. Delete
3. Display
4. Exit
Enter choice: 3
Queue: 0 5
```

UNIT 3:

5. A department maintains student roll numbers dynamically. Write a C program using a Singly Linked List to create the list, insert at the beginning and end, search for a specified roll number, delete a specified roll number, and display the updated list after each operation. Handle the case when a requested roll number is not available.

Source code :-

```
#include <stdio.h>

#include <stdlib.h>

struct Node

{

   int roll;

   struct Node *next;

};

struct Node *head = NULL;

void create()

{

   int n, roll, i;

   struct Node *newNode, *temp;

   printf("Enter number of students: ");

   scanf("%d", &n);

   for (i = 0; i < n; i++)

   {

      printf("Enter roll number: ");

      scanf("%d", &roll);

      newNode = malloc(sizeof(struct Node));

      newNode->roll = roll;

      newNode->next = NULL;

      if (head == NULL)

      {

         head = newNode;

      }

      else {
```

```
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = newNode;
        }
    }
}
void insertBeginning()
{
    int roll;
    struct Node *newNode;
    printf("Enter roll number: ");
    scanf("%d", &roll);
    newNode = malloc(sizeof(struct Node));
    newNode->roll = roll;
    newNode->next = head;
    head = newNode;
    printf("Inserted at beginning\n");
}
void insertEnd()
{
    int roll;
    struct Node *newNode, *temp;
    printf("Enter roll number: ");
    scanf("%d", &roll);
    newNode = malloc(sizeof(struct Node));
    newNode->roll = roll;
    newNode->next = NULL;
    if (head == NULL)
        head = newNode;
    else {
```

```
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    printf("Inserted at end\n");
}
void search()
{
    int roll, pos = 1;
    struct Node *temp = head;
    printf("Enter roll number to search: ");
    scanf("%d", &roll);
    while (temp != NULL)
    {
        if (temp->roll == roll)
        {
            printf("Roll number found at position %d\n", pos);
            return;
        }
        temp = temp->next;
        pos++;
    }
    printf("Roll number not found\n");
}
void deleteNode()
{
    int roll;
    struct Node *temp, *prev;
    printf("Enter roll number to delete: ");
    scanf("%d", &roll);
```

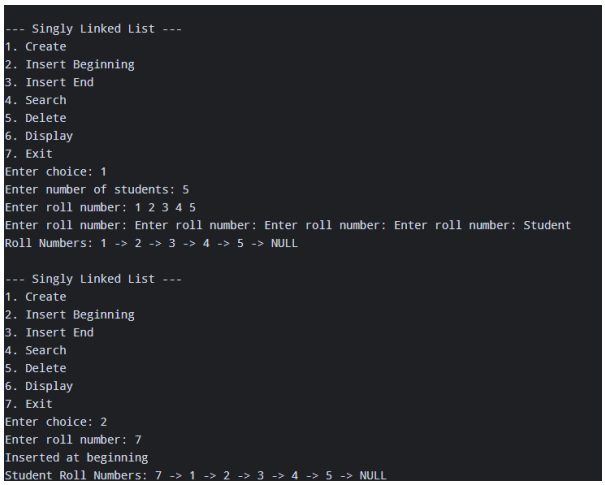
```
    temp = head;
    prev = NULL;
    while (temp != NULL && temp->roll != roll)
    {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL)
    {
        printf("Roll number not found\n");
        return;
    }
    if (prev == NULL)
        head = temp->next;
    else
        prev->next = temp->next;
    free(temp);
    printf("Roll number deleted\n");
}
void display()
{
    struct Node *temp = head;
    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }
    printf("Student Roll Numbers: ");

    while (temp != NULL)
```

```
    {
        printf("%d -> ", temp->roll);
        temp = temp->next;
    }
    printf("NULL\n");
}
int main()
{
    int choice;
    while (1)
    {
        printf("\n--- Singly Linked List ---\n");
        printf("1. Create\n");
        printf("2. Insert Beginning\n");
        printf("3. Insert End\n");
        printf("4. Search\n");
        printf("5. Delete\n");
        printf("6. Display\n");
        printf("7. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                create();
                display();
                break;
            case 2:
                insertBeginning();
                display();
                break;
```

```
            case 3:
                insertEnd();
                display();
                break;
            case 4:
                search();
                break;
            case 5:
                deleteNode();
                display();
                break;
            case 6:
                display();
                break;
            case 7:
                return 0;
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}
```

Output :-



6. Develop a C program for a Doubly Linked List representing a sequence of web pages visited by a user. The program should insert a new page, move forward and backward, delete a specified page, and display the pages from first-to-last and last-to-first while handling beginning and end conditions correctly.

Source code :-

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

struct Node

{

   char page[50];

   struct Node *prev;

   struct Node *next;

};

struct Node *head = NULL;

struct Node *current = NULL;

void insertPage()

{

   char page[50];

   struct Node *newNode;

   printf("Enter page name: ");

   scanf("%s", page);

   newNode = malloc(sizeof(struct Node));

   strcpy(newNode->page, page);

   newNode->prev = NULL;

   newNode->next = NULL;

   if (head == NULL)

   {

      head = newNode;

      current = newNode;

   }

   else
```

```
    {
        current->next = newNode;
        newNode->prev = current;
        current = newNode;
    }
    printf("Page inserted\n");
}
void moveForward()
{
    if (current == NULL)
        printf("No pages available\n");
    else if (current->next == NULL)
        printf("Already at the last page\n");
    else
    {
        current = current->next;
        printf("Current page: %s\n", current->page);
    }
}
void moveBackward()
{
    if (current == NULL)
        printf("No pages available\n");
    else if (current->prev == NULL)
        printf("Already at the first page\n");
    else
    {
        current = current->prev;
        printf("Current page: %s\n", current->page);
    }
}
```

```
void deletePage()
{
   char page[50];
   struct Node *temp;
   printf("Enter page to delete: ");
   scanf("%s", page);
   temp = head;
   while (temp != NULL && strcmp(temp->page, page) != 0)
      temp = temp->next;
   if (temp == NULL)
   {
      printf("Page not found\n");
      return;
   }
   if (temp->prev != NULL)
      temp->prev->next = temp->next;
   else
      head = temp->next;
   if (temp->next != NULL)
      temp->next->prev = temp->prev;
   if (current == temp)
   {
      if (temp->next != NULL)
         current = temp->next;
      else
         current = temp->prev;
   }
   free(temp);
   printf("Page deleted\n");
}
```

```
void displayForward()
{
   struct Node *temp = head;
   if (head == NULL)
   {
      printf("No pages available\n");
      return;
   }
   printf("Pages (First to Last): ");
   while (temp != NULL)
   {
      printf("%s <-> ", temp->page);
      temp = temp->next;
   }
   printf("NULL\n");
}
void displayBackward()
{
   struct Node *temp = head;
   if (head == NULL)
   {
      printf("No pages available\n");
      return;
   }
   while (temp->next != NULL)
      temp = temp->next;
   printf("Pages (Last to First): ");
   while (temp != NULL)
   {
      printf("%s <-> ", temp->page);
```

```
        temp = temp->prev;
    }
    printf("NULL\n");
}
int main()
{
    int choice;
    while (1)
    {
        printf("\n--- Web Page History ---\n");
        printf("1. Insert Page\n");
        printf("2. Move Forward\n");
        printf("3. Move Backward\n");
        printf("4. Delete Page\n");
        printf("5. Display First to Last\n");
        printf("6. Display Last to First\n");
        printf("7. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                insertPage();
                break;
            case 2:
                moveForward();
                break;
            case 3:
                moveBackward();
                break;
            case 4:
```

```
            deletePage();
            break;
        case 5:
            displayForward();
            break;
        case 6:
            displayBackward();
            break;
        case 7:
            return 0;
        default:
            printf("Invalid choice\n");
      }
   }
   return 0;
}
```

Output :-

```
--- Web Page History ---
1. Insert Page
2. Move Forward
3. Move Backward
4. Delete Page
5. Display First to Last
6. Display Last to First
7. Exit
Enter choice: 1
Enter page name: aswanth
Page inserted

--- Web Page History ---
1. Insert Page
2. Move Forward
3. Move Backward
4. Delete Page
5. Display First to Last
6. Display Last to First
7. Exit
Enter choice: 1
Enter page name: sai
Page inserted

--- Web Page History ---
1. Insert Page
2. Move Forward
3. Move Backward
4. Delete Page
5. Display First to Last
6. Display Last to First
7. Exit
Enter choice: 5
Pages (First to Last): aswanth <-> sai <-> NULL
```

UNIT 4 :

7. A system stores unique integer identification numbers using a Binary Search Tree. Write a C program to insert n values, display inorder, preorder and postorder traversals, search for a specified value, and report whether it exists. Use the output to explain why inorder traversal produces sorted values.

Source code :-

```
#include <stdio.h>

#include <stdlib.h>

struct Node

{

   int data;

   struct Node *left;

   struct Node *right;

};

struct Node* createNode(int value){

   struct Node *newNode;

   newNode = (struct Node*)malloc(sizeof(struct Node));

   newNode->data = value;

   newNode->left = NULL;

   newNode->right = NULL;

   return newNode;

}

struct Node* insert(struct Node *root, int value)

{

   if (root == NULL)

      return createNode(value);

   if (value < root->data)

      root->left = insert(root->left, value);

   else if (value > root->data)

      root->right = insert(root->right, value);

   return root;

}
```

```
void inorder(struct Node *root)
{
   if (root != NULL)
   {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}
void preorder(struct Node *root)
{
   if (root != NULL)
   {
      printf("%d ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}
void postorder(struct Node *root)
{
   if (root != NULL)
   {
      postorder(root->left);
      postorder(root->right);
      printf("%d ", root->data);
   }
}
int search(struct Node *root, int value)
{
   if (root == NULL)
```

```
        return 0;
    if (root->data == value)
        return 1;
    if (value < root->data)
        return search(root->left, value);
    else
        return search(root->right, value);
}
int main()
{
    struct Node *root = NULL;
    int n, value, key, i;
    printf("Enter number of values: ");
    scanf("%d", &n);
    printf("Enter values:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &value);
        root = insert(root, value);
    }
    printf("\nInorder Traversal: ");
    inorder(root);
    printf("\nPreorder Traversal: ");
    preorder(root);
    printf("\nPostorder Traversal: ");
    postorder(root);
    printf("\n\nEnter value to search: ");
    scanf("%d", &key);
if(search(root, key))
        printf("Value %d exists in BST\n", key);
    else
```

```
        printf("Value %d does not exist in BST\n", key);
    return 0;
}
}
```

Output :-

```
Enter number of values: 5
Enter values:
10 14 5 12 13

Inorder Traversal: 5 10 12 13 14
Preorder Traversal: 10 5 14 12 13
Postorder Traversal: 5 13 12 14 10

Enter value to search: 5
Value 5 exists in BST
```

8. Extend a Binary Search Tree program to support deletion. The program should create a BST, delete a user-specified node, correctly handle nodes with zero, one and two children, and display inorder traversal before and after deletion. Test the program separately for all three deletion cases.

Source code :-

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
struct Node* createNode(int value)
{
    struct Node *newNode;
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
```

```
struct Node* insert(struct Node *root, int value)
{
   if (root == NULL)
      return createNode(value);
   if (value < root->data)
      root->left = insert(root->left, value);
   else if (value > root->data)
      root->right = insert(root->right, value);
   return root;
}
void inorder(struct Node *root)
{
   if (root != NULL)
   {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}
struct Node* findMin(struct Node *root)
{
   while (root->left != NULL)
      root = root->left;
   return root;
}
struct Node* deleteNode(struct Node *root, int key)
{
   struct Node *temp;
   if (root == NULL)
      return root;
```

```
    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else
{       /* Case 1: No children */
        if (root->left == NULL && root->right == NULL)
        {
            free(root);
            return NULL;
        }
        /* Case 2: One child */
        if (root->left == NULL)
        {
            temp = root->right;
            free(root);
            return temp;
        }
        if (root->right == NULL)
        {
            temp = root->left;
            free(root);
            return temp;
        }
        /* Case 3: Two children */
        temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
```

```
    return root;
}

int main()
{
    struct Node *root = NULL;
    int n, value, key, i;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter values:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &value);
        root = insert(root, value);
    }
    printf("\nInorder before deletion: ");
    inorder(root);
    printf("\nEnter node to delete: ");
    scanf("%d", &key);
    root = deleteNode(root, key);
    printf("\nInorder after deletion: ");
    inorder(root);
    printf("\n");
    return 0;
}
```

Output :-

